



Demo API Endpoints for AI Infrastructure

A guide to creating simple, stable, and explainable endpoints that showcase your AI system in action for both technical and non-technical stakeholders.

Why Create Demo Endpoints?

Not all stakeholders can interpret logs or metrics directly. A working endpoint provides concrete proof that your entire stack works together:

Demonstrates **infrastructure + model + monitoring** functioning together

Enables **hands-on testing** by reviewers with minimal technical knowledge

Serves as a **safe public-facing showcase** for your capabilities



Qualities of a Good Demo Endpoint

1

Simple Contract

Limited to 1-2 inputs with clear JSON schema that's easy to understand

2

Fast Responses

Latency under 1 second to maintain engagement during live demos

3

Safe Outputs

Filter PII and avoid offensive content to ensure professional demos

4

Stable Version

Pin specific model version dedicated to demo environment

5

Monitored

Include metrics, logs, and rate limits for observability

Example: Sentiment Analysis API

Request:

```
POST /predict
Content-Type: application/json
{
  "text": "I love this course!"
}
```

Response:

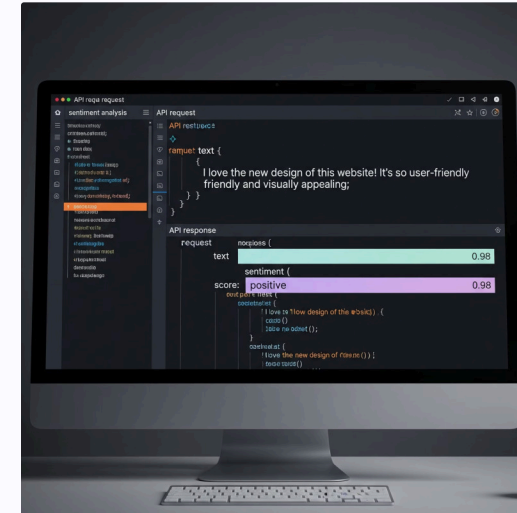
```
{
  "model": "distilbert-qa-v1.3",
  "label": "POSITIVE",
  "score": 0.94,
  "latency_ms": 112
}
```

Why This Works:

Shows clear **input** → **inference** → **output** flow

Includes helpful **metadata** (model version, latency)

Simple enough for non-technical stakeholders



Health & Metadata Endpoints

/healthz

Simple 200 OK response for kubernetes probes and basic availability checks

/readyz

Deeper health check verifying dependencies (feature store, DB, GPU availability)

/metadata

Returns model info, version, SLOs, and uptime statistics

Example Metadata Response:

```
{  
  "service": "capstone-infer",  
  "version": "v1.3",  
  "uptime": "48h",  
  "gpu_utilization": "0.42"  
}
```

Demo-Friendly Enhancements

Interactive Swagger/ReDoc UI

Auto-generated from OpenAPI spec, allows testing directly from browser

Playground UI

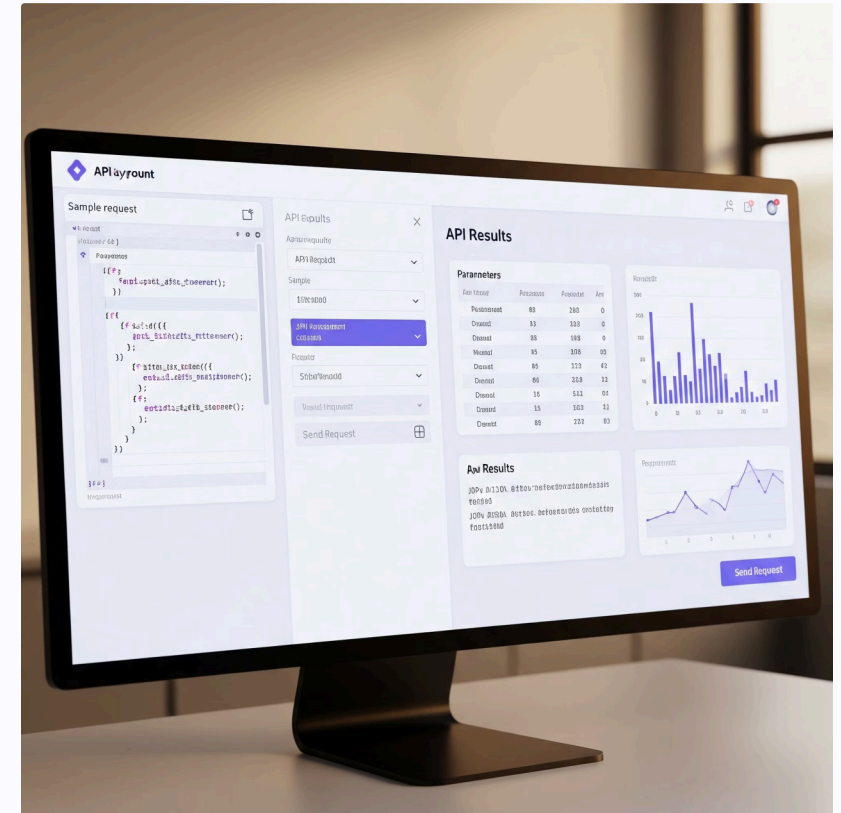
Custom web form calling /predict with friendly interface for non-technical users

Pre-baked Examples

"Click here to test fraud detection" buttons with sample inputs

Explainability Add-ons

Return top features or attention weights to explain model decisions





Security & Limits for Demo Endpoints



API Keys or Demo Tokens

Time-bound credentials for controlled access, especially for external reviewers



Rate Limiting

Implement 5 req/sec per user limit to prevent resource abuse



Isolation

Deploy in sandbox namespace completely isolated from production systems



Protection Mechanisms

Add watermarking or use stub models if endpoint will be publicly accessible

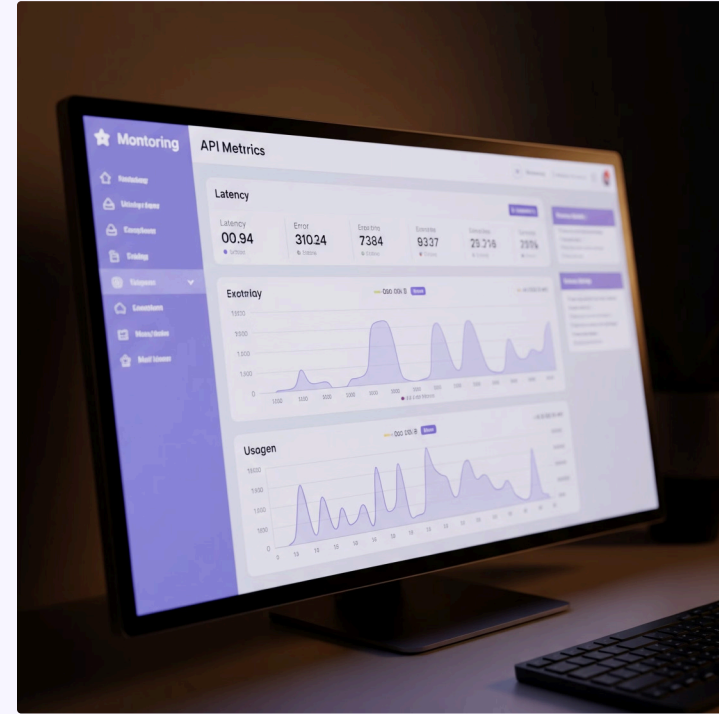
Observability for Demo Endpoints

Key Metrics to Track:

- Latency (p50, p95, p99)
- Error percentage
- Token usage
- Cost per 1,000 requests
- Request volume by endpoint

Set Up Alerts For:

- Abuse patterns (high QPS)
- Elevated error rates
- Token expiry warnings



Pro Tip: Create a dedicated "Demo Traffic" Grafana dashboard and link it directly to your Capstone presentation for live monitoring during reviews.

Deployment Patterns



Blue/Green

One stable demo version with zero-downtime updates



Namespace Isolation

demo-infer completely separate from prod environment



Canary

Showcase new features on limited traffic subset



Ephemeral Demos

Spin up on demand via Helm chart/ArgoCD



Code Example (FastAPI Demo)

```
from fastapi import FastAPI
from pydantic import BaseModel
import time, random

app = FastAPI(title="Capstone Demo API")

class Inp(BaseModel):
    text: str

@app.post("/predict")
def predict(inp: Inp):
    start = time.time()
    label = random.choice(["POSITIVE", "NEGATIVE"])
    score = round(random.uniform(0.6, 0.99), 3)
    latency = round((time.time() - start) * 1000, 2)

    return {
        "model": "demo-v1",
        "label": label,
        "score": score,
        "latency_ms": latency
    }
```

📄 🙌 **Replace the random choice with your actual model inference logic.**

This minimal example:

- Defines a schema for inputs
- Simulates model prediction
- Tracks actual latency
- Returns structured response

FastAPI automatically generates OpenAPI documentation for this endpoint.

Stakeholder Walkthrough

- 1 — Start with Swagger UI**
Show the interactive documentation that demonstrates available inputs and expected outputs
- 2 — Demo Live Monitoring**
Walk through Grafana dashboards showing real-time latency and error percentage metrics
- 3 — Conduct Failover Drill**
Kill a pod in real-time to demonstrate how the API stays up through redundancy
- 4 — Review Drift Reports**
Show overlay reports that highlight when input patterns change unexpectedly



Polishing for Demo Day

Documentation

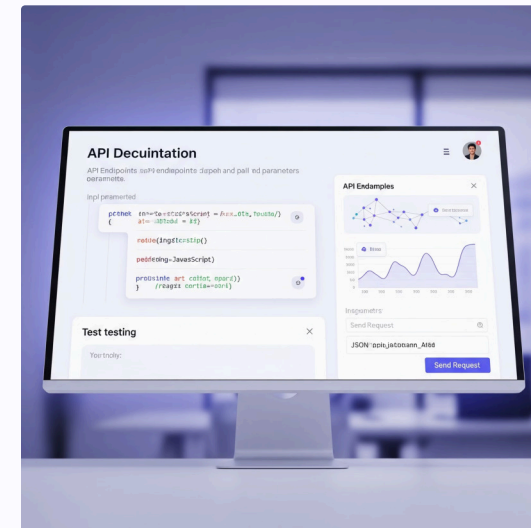
- Add friendly documentation at `/docs` (Swagger)
- Create detailed guides at `/help` (Markdown)
- Pre-load sample payloads for easy stakeholder testing

Visualization

- Design colorful Grafana dashboard specifically for demo traffic
- Create visualization widgets for model outputs when appropriate

Backup Plans

- Prepare local curl commands if API fails
- Have Jupyter notebook demo ready as fallback
- Record short video of successful demo for worst-case scenarios



Capstone Deliverables

1 Stable /predict Endpoint

Fully functional inference API with sample request templates

2 Health Check Suite

/healthz, /readyz, and /metadata endpoints implemented and tested

3 API Documentation

OpenAPI specification published with interactive Swagger/ReDoc interface

4 Monitoring Dashboard

Dedicated "demo view" Grafana dashboard showing key metrics

5 Security Controls

Demo API secured with appropriate authentication and rate-limiting

Summary

Demo endpoints are living proof of infrastructure success

They must be:



Simple

Clear inputs and outputs that anyone can understand



Stable

Reliable performance with consistent behavior



Safe

Protected against misuse and inappropriate content



Explainable

Transparent about how decisions are made

A well-designed demo endpoint makes your Capstone project **impressive for stakeholders** by giving them something tangible to interact with.





Discussion Question

Would you expose your Capstone demo as a public sandbox or keep it internal-only with tokens?

Public Sandbox Considerations

- Wider feedback and testing
- Demonstrates confidence in implementation
- Could attract interest from potential employers
- May require additional abuse protection

Internal Token-Based Considerations

- More controlled testing environment
- Protection of proprietary techniques
- Reduced risk of resource abuse
- Easier to manage expectations